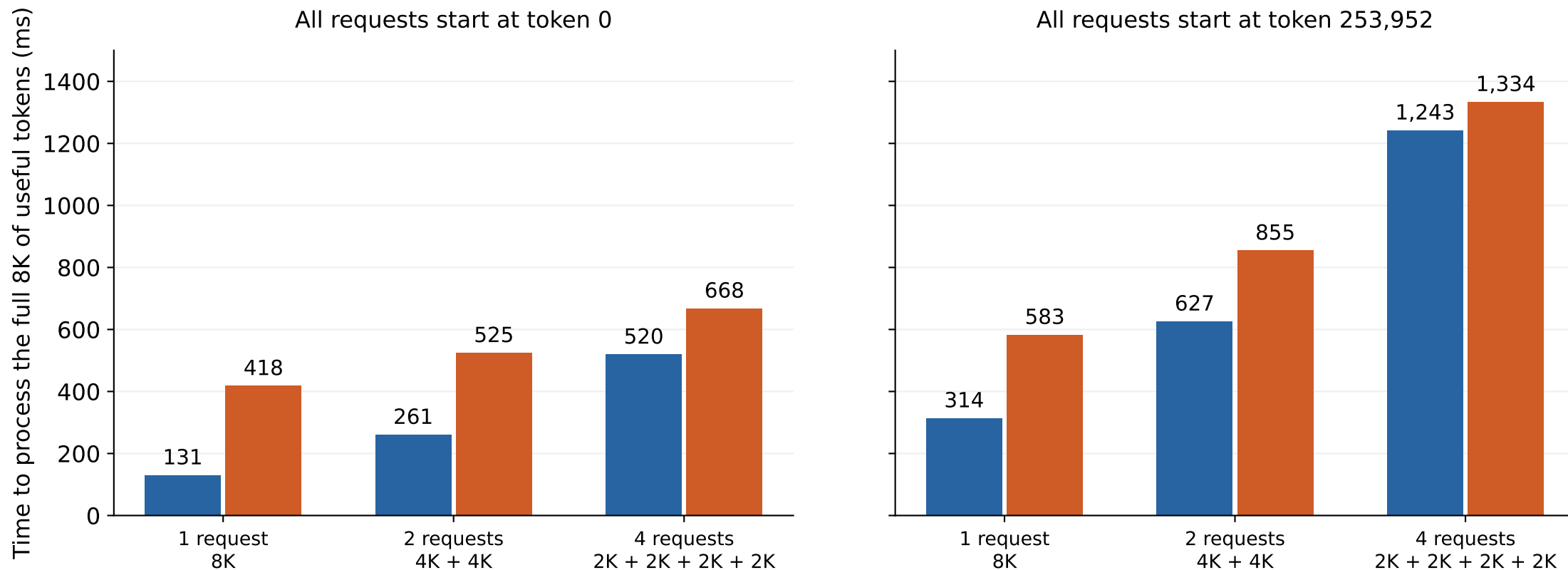


8K chunked prefill vs full 8K ragged batches

Every ragged batch is FULL: 8,192 useful tokens. Lower time is better.

■ Chunked prefill (8K) ■ Ragged attention (full 8K batch)



Question: Does dividing a full 8K batch across more requests change which method is faster?

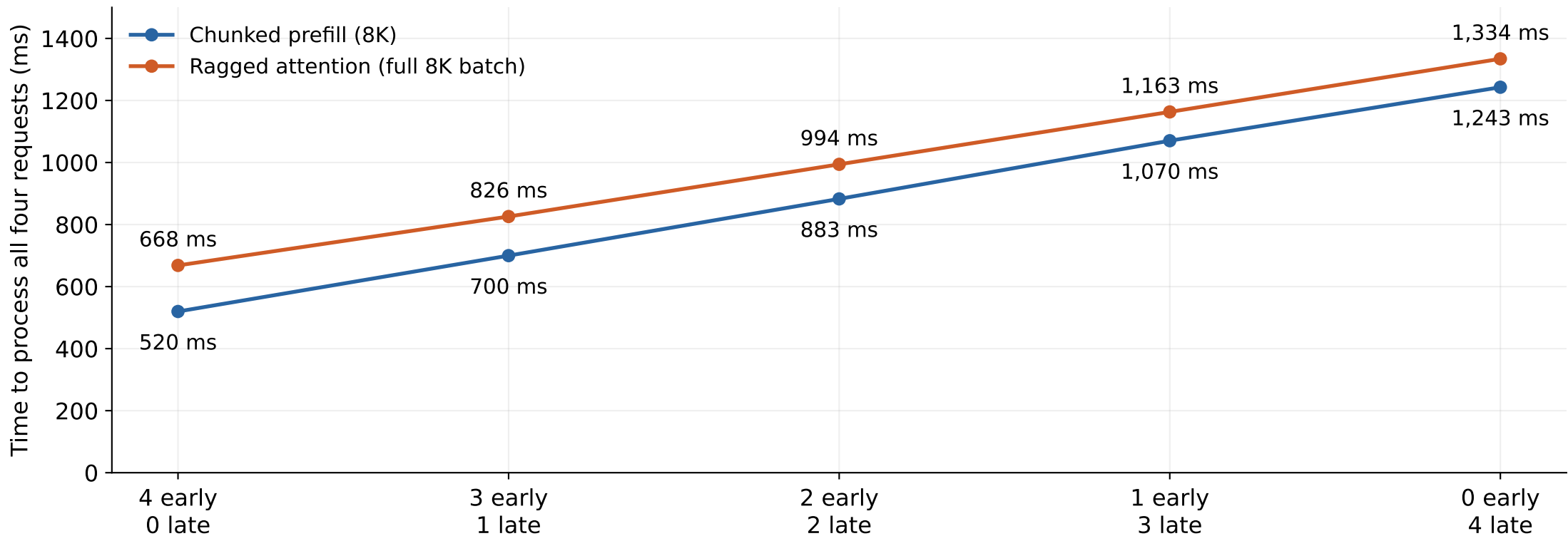
For 1 / 2 / 4 requests, chunked uses 1 / 2 / 4 separate 8K calls; ragged uses one full 8K call. Each pair of bars processes the same useful tokens for the same requests. Start index = tokens already processed for that request.

Result: Chunked prefill is faster in all six cases. For four 2K requests, ragged takes 29% more time at the beginning and 7% more time near the end.

K = 1,024 tokens. Time means completion of ALL requests shown, not time per request. 60 layers; CP8/TP4; median of five warmed runs, including staging and synchronization. Initial compilation/capture is excluded. Values reuse the completed 2026-10-08 measurements. In the current implementation, request pieces shorter than 8K must be final pieces.

Four requests: mostly beginning, mostly end, or mixed

Every ragged batch is FULL: 8,192 useful tokens. Lower time is better.



Question: With four requests contributing 2,048 tokens each, what happens as more have long histories?

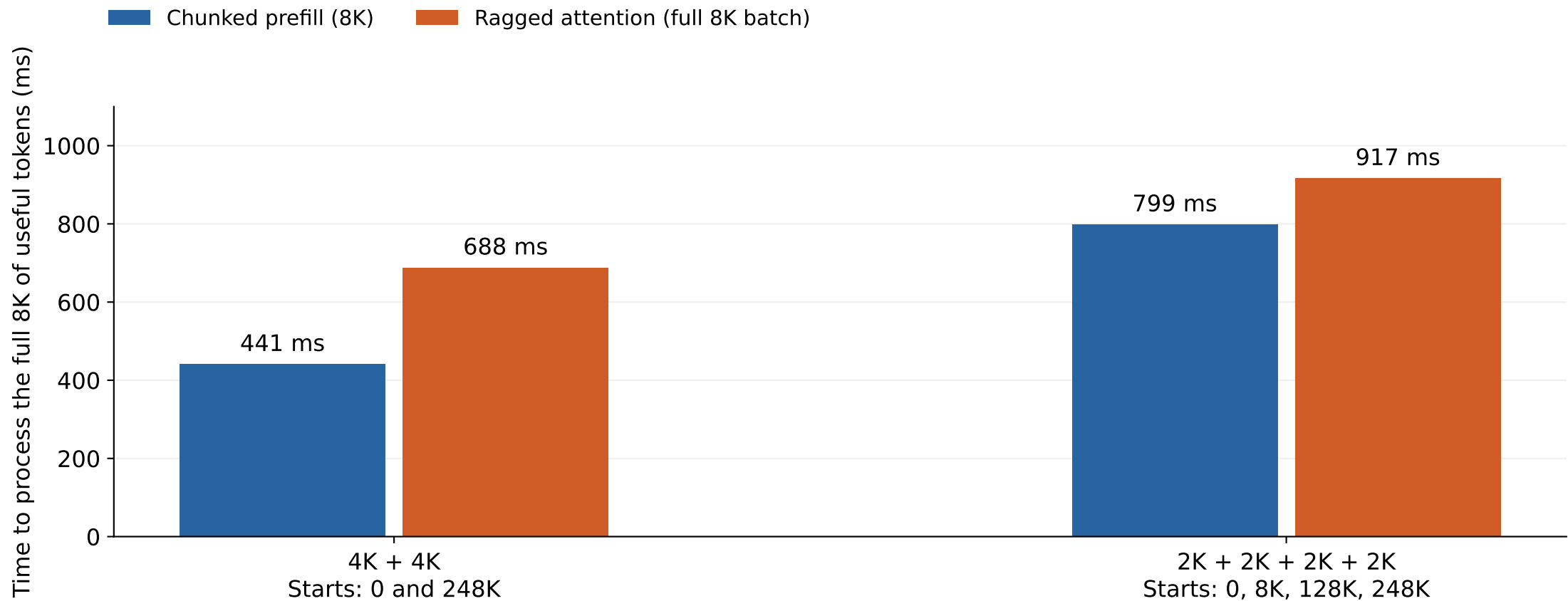
Every point is $4 \times 2,048 = 8,192$ useful tokens. Early requests start at 0; late requests start at 253,952. Every request's end index is its start index + 2,048.

Result: Both methods slow down as more requests have long histories. Ragged remains slower. Its extra time is 29%, 18%, 13%, 9%, 7% as the number of late requests increases from zero to four.

K = 1,024 tokens. Time means completion of ALL requests shown, not time per request. 60 layers; CP8/TP4; median of five warmed runs, including staging and synchronization. Initial compilation/capture is excluded. Values reuse the completed 2026-10-08 measurements. In the current implementation, request pieces shorter than 8K must be final pieces.

One full 8K batch, requests at different positions

Every ragged batch is FULL: 8,192 useful tokens. Lower time is better.



Question: How do the two methods compare when requests in the batch have different amounts of history?

Two-request ranges: [0, 4,096), [253,952, 258,048). Four-request ranges: [0, 2,048), [8,192, 10,240), [131,072, 133,120), [253,952, 256,000).
End indices are exclusive.

Result: Ragged takes 56% more time for the two-request batch and 15% more time for the four-request batch. Mixing positions does not produce a speedup here.

K = 1,024 tokens. Time means completion of ALL requests shown, not time per request. 60 layers; CP8/TP4; median of five warmed runs, including staging and synchronization. Initial compilation/capture is excluded. Values reuse the completed 2026-10-08 measurements. In the current implementation, request pieces shorter than 8K must be final pieces.